

Tóm tắt điều hành

Phát hiện buồn ngủ thời gian thực từ khuôn mặt cần mạng CNN rất nhẹ (<50k tham số) kết hợp với các đặc trưng hình ảnh như chớp mắt, ngáp, nghiêng đầu,... Các hệ thống thông thường gồm: phát hiện mặt (ví dụ MTCNN/Haar cascade), định vị điểm mốc (landmarks, ví dụ MediaPipe), rồi tính các chỉ số như *EAR* (Eye Aspect Ratio), *MAR* (Mouth Aspect Ratio) để xác định trạng thái nhắm mắt, ngáp. Dữ liệu huấn luyện tiêu biểu gồm các bộ *NTHU-DDD* (video lái xe giả buồn ngủ), *UTA-RLDD* (video ba mức trạng thái), *YawDD* (dữ liệu ngáp), *CEW* (ảnh mắt đóng/mở) ¹ ². Để chạy thời gian thực trên MCU, cần tối ưu mạng: cắt tỉa (pruning), lượng tử hóa (quantization) xuống 8-bit hoặc 4-bit ³, chia sẻ trọng số, lớp tích chập tách riêng (depthwise conv) hoặc thậm chí nhị phân hóa. Kết hợp huấn luyện tri thức (knowledge distillation) từ mạng lớn và tìm kiếm trúc tự động (NAS) giúp tạo mô hình dưới 50k tham số. Ví dụ, *DDD TinyML* đã đạt kích thước ~50KB với mạng CNN tùy chỉnh khi sử dụng lượng tử hóa động (DRQ) ³. Độ trễ mục tiêu: vài chục ms trên CPU ARM (Cortex-A), vài ms với NPU (như EdgeTPU), hay chục ms trên MCU tối tân (STM32H7) với mô hình cực nhỏ. Cuối cùng, cần xem xét độ riêng tư (xử lý trên thiết bị), giới hạn ứng dụng (độ tin cậy kém khi ánh sáng xấu, tài xế đeo kính), và nguy cơ báo động giả.

1. Các mô hình nhẹ (<50k tham số) và so sánh

Dưới đây liệt kê một số kiến trúc CNN vi mô và mô hình TinyML có thể áp dụng. Các giá trị tham số và kích thước **chiều vào** (pixels) chỉ mang tính ước lượng; độ trễ *latency* phụ thuộc phần cứng và tối ưu. Độ chính xác tham khảo là trên các nhiệm vụ nhận dạng dấu hiệu buồn ngủ (chớp mắt, ngáp) nếu có.

Mô hình	Tham số	Kích thước input	Độ trễ ví dụ (CPU/ARM/NPU)	Độ chính xác nổi bật	Pretrained	License	Ưu/Nhược điểm
TinyCNN (tùy chỉnh)	~30–50K	(32×32) up to (64×64)	~20–50ms (ARM Cortex-A), ~0.5ms (EdgeTPU)**	~95–99% (có thể trên tập con	Không	Tùy ý	+ Cực nhỏ, có thể thiết kế riêng cho mắt/miệng. – Độ chính xác giới hạn, khó học được chi tiết phức tạp nếu quá nhỏ.
SqueezeNet (pruned/quantized)	~1.2M (gốc) nhưng ~35K tham số sau tinh chỉnh**	224×224 (gốc)	~80ms (ARM), ~10ms (EdgeTPU)	≈98–99% (drowsiness)*	Có (gốc ImageNet)	Apache 2.0	+ Nhỏ gốc, kiến trúc Fire module; có phiên bản pruned xuống vài chục KB ³ . – Còn lớn gốc, cần prune/quantize mạnh.

Mô hình	Tham số	Kích thước input	Độ trễ ví dụ (CPU/ARM/NPU)	Độ chính xác nổi bật	Pretrained	License	Ưu/Nhược điểm
MobileNet-V2 (siêu nhỏ)	>100k (thông thường) nhưng có thể <i>distill</i> xuống <50k	96×96 (proposed)	~100ms (ARM), ~~2ms (EdgeTPU)**	99.6% (DDD TinyML report) ⁴	Có (ImageNet)	Apache 2.0	+ Depthwise separable, tiết kiệm. - Dễ pruned/quantize. - Độ trễ có thể thấp với NN accelerators. - Cần giảm kích thước (nhân tử độ sâu) mạnh để <50k.
MobileNet-V3 Small	~65k (gốc)	(128×128) up to 224×224	~50ms (ARM), ~~5ms (EdgeTPU)**	~98-99% (như trên)	Có (ImageNet)	Apache 2.0	+ Cải tiến từ V2, hiệu năng cao. - Tuy vậy gốc vẫn ~65k, cần distill/prune tiếp. - Latency thấp trên NPU.
ESPNet Lite / ShuffleNetV2	~100k (gốc)	224×224	~60ms (ARM)	Không cụ thể	Có	Apache 2.0	+ Thiết kế siêu nhẹ (ShuffleNetV2), Depthwise conv (ESPNet). - Gốc >50k, nhưng có thể cắt giảm sâu và lượng tử hóa.

Mô hình	Tham số	Kích thước input	Độ trễ ví dụ (CPU/ARM/NPU)	Độ chính xác nổi bật	Pretrained	License	Ưu/Nhược điểm
TinyML Speech-CNN (ví dụ trên STM32)	~20–30K	64×64 (thường)	~20ms (Cortex-M4)	~95% (trong ngữ cảnh khác)	Không	MIT? (tùy)	+ Ví dụ: các mô hình nhận dạng tiếng nói hoặc âm thanh trên MCU (TensorFlow Lite Micro) chỉ ~20–30K tham số. – Cần chuyển đổi cấu trúc cho ảnh.
Conv+LSTM/ 1D-CNN nhỏ	~~30K	n/a (dãy features)	~~10ms (ARM)	–	–	–	+ Mô hình đánh giá theo chuỗi (đo EAR qua thời gian) như LSTM nhỏ hay 1D-CNN giúp phát hiện xu hướng dần. – Khó train nếu quá nhỏ.

Cột độ trễ chỉ mang tính ví dụ chung: với **ARM Cortex-A** (như Pi hoặc smartphone) thường vài chục ms; **NPU vi xử lý** (EdgeTPU, Neural Engine) có thể giảm xuống ~1–5ms; **MCU** (như STM32H7) yêu cầu model cực nhỏ mới đáp ứng được, latency chục ms.

Ghi chú: Độ chính xác trên tác vụ liên quan đến buồn ngủ ước tính từ các báo cáo (DDD TinyML, nêu 99% với MobileNet-V2) ⁴. “*” Latency EdgeTPU giả định mô hình được chuyển về TensorFlow Lite 8-bit, đem ra TPU.

2. Đặc trưng hình ảnh trong phát hiện buồn ngủ

Các hệ thống dựa vào camera thường sử dụng nhiều đặc trưng **hành vi** trên khuôn mặt để phát hiện buồn ngủ. Nghiên cứu tổng hợp chỉ ra các đặc trưng quan trọng gồm:

- **Mí mắt (nhắm/mở):** Tỷ lệ mí mắt nhắm (khóa mắt) và **EAR (Eye Aspect Ratio)** được tính từ tọa độ landmarks quanh mắt ⁵. Khi mí mắt nhắm, EAR giảm xuống dưới ngưỡng (thường ~0.18 ⁵). Chớp mắt nhiều hơn/bị nhắm lâu hơn là tín hiệu buồn ngủ ⁶ ⁷. Ví dụ, PERCLOS (tỷ lệ thời gian mí mắt nhắm $\geq 80\%$) là chỉ số nổi bật để đánh giá buồn ngủ ⁶.
- **Tần số nháy mắt:** Người mệt thường nháy mắt khác biệt; mô hình phân tích dãy nhịp chớp (eye-blink) qua thời gian giúp dự đoán mức cảnh giác ⁸. Giảm tần số hoặc tăng độ dài nháy là dấu hiệu cảnh báo ⁸.

- **Ngáp:** Mở miệng (yawning) là phản xạ điển hình khi mệt mỏi. Nhiều hệ thống dùng **Mouth Aspect Ratio** hoặc CNN để nhận dạng ngáp ⁹. Nghiên cứu cho thấy kết hợp tín hiệu ngáp và chớp mắt tăng độ chính xác hơn so với chỉ dùng chớp mắt ¹⁰.
- **Góc đầu (head pose) và gật đầu:** Khi buồn ngủ, tài xế thường cúi gập đầu hoặc gật đầu (micro-nod) do nặng đầu ¹¹. Theo [36], “Khi mệt, đầu người lái thường ngả về phía trước rồi gật lên do mất trương lực” ¹¹. Theo dõi đầu (ví dụ POSIT, IPPE) và nhận dạng hành vi gật đầu cho dữ liệu thời gian thực.
- **Hướng nhìn (gaze):** Ngoài ra, theo dõi ánh mắt/dòng nhìn có thể phát hiện nếu người lái không tập trung (nhìn sang bên). Chẳng hạn, “hệ thống mới sử dụng kết hợp hướng nhìn và trạng thái chớp mắt để đánh giá buồn ngủ” ¹².
- **Biểu cảm cơ mặt (Action Units):** Một số nghiên cứu dùng **Facial Action Coding System (FACS)** để mã hoá cơ mặt; ví dụ AU “nâng mí mắt” (Upper Lid Raiser) hay “hạ cằm” (Jaw Drop) có tương quan với buồn ngủ ¹³ ¹⁴. Phương pháp này đạt độ chính xác cao ($\approx 99\%$ trên NTHU DDD) và cho biết rõ ràng các cơ mặt đóng góp nào ¹³ ¹⁴.
- **Biểu cảm vi tế (Micro-expressions):** Các nét biểu cảm rất nhẹ (nháy mắt chậm, cau mày...) có thể xuất hiện khi bắt đầu buồn ngủ. Bộ dữ liệu UTA-RLDD nhấn mạnh nhận dạng các vi-biểu cảm này để phát hiện giai đoạn đầu của buồn ngủ ¹⁵. Kết quả cho thấy dùng ảnh chỉ vùng mắt (thu hẹp) giúp tăng tốc độ và độ chính xác so với dùng cả mặt ¹⁶.
- **Nhiệt độ và màu da:** Mặc dù không phải tín hiệu hình ảnh trực tiếp, một số nghiên cứu dùng **camera nhiệt** để đo nhiệt độ trán, má tài xế. Chứng minh: khi buồn ngủ, nhiệt độ trán giảm khoảng 0.46°C , chênh lệch nhiệt giữa má và trán giảm $\sim 0.81^{\circ}\text{C}$ ¹⁷ ¹⁸. Tóm lại, “nhiệt độ trán và gradient má-trán giảm khi tài xế buồn ngủ” ¹⁷ ¹⁸. Tuy vậy, cần camera hồng ngoại chuyên dụng, không phổ biến trên thiết bị biên thông thường.
- **Cân nhắc đa nguồn:** Nhiều hệ thống phối hợp các đặc trưng trên khuôn mặt (mắt + miệng + đầu) cho kết quả tốt hơn dùng đơn lẻ. Ví dụ, một công trình cho thấy dùng **cả ảnh mặt và ảnh đôi mắt** kết hợp (fusional) cho độ chính xác cao nhất ¹⁶. Nhóm nghiên cứu tại Berkeley kết luận “xem xét đầy đủ các đặc trưng khuôn mặt có thể gia tăng đáng kể độ chính xác” so với chỉ dùng chớp mắt ¹⁹.

Tóm lại, bộ đặc trưng đa dạng gồm hình thái mắt (EAR/PERCLOS), tần số chớp, hành vi miệng (yawning), dáng đầu/góc nhìn và các cơ mặt được dùng rộng rãi. Trong thực tế triển khai thiết bị biên, có thể chọn những đặc trưng dễ đo nhất (EAR, MAR) và đơn giản hóa bằng ngưỡng hoặc mô hình chuỗi nhỏ (1D-CNN/LSTM), tránh dùng các phép toán phức tạp như phân tích vi-biểu cảm đòi hỏi dữ liệu nhạy.

3. Quy trình triển khai thời gian thực

Một pipeline phổ biến để phát hiện buồn ngủ gồm các bước chính:

flowchart TD

```

A[Video từ camera] --> B[Phát hiện khuôn mặt]
B --> C[Định vị landmarks khuôn mặt]
C --> D[Tính toán đặc trưng tức thời: EAR, MAR, góc đầu,...]
D --> E[Tổng hợp đặc trưng qua thời gian (ví dụ EMA, 1D-CNN/LSTM)]
E --> F[Phân lớp/Ngưỡng: tỉnh/tương/mơ màng]
F --> G[Phát cảnh báo nếu buồn ngủ]

```

- **Phát hiện mặt:** Sử dụng các phương pháp như Haar cascade, HOG+SVM, MTCNN (Multi-task CNN) để tìm vị trí khuôn mặt. Tốc độ nhanh, độ chính xác cao rất cần thiết. Nhiều nghiên cứu dùng MTCNN vì vừa phát hiện mặt lẫn landmarks ²⁰ ²¹.

- **Định vị landmarks:** Sau khi cắt vùng mặt, dùng bộ dò landmarks (ví dụ *dlib*, *MediaPipe FaceMesh*, *OpenCV Facemark*) để xác định các điểm xung quanh mắt, mũi, miệng. Những điểm này cho phép tính EAR/MAR, góc đầu (qua mắt/mũi/miệng), và nhận dạng ngáp (qua độ mở miệng). Một ví dụ điển hình là sử dụng **MediaPipe** để lấy tọa độ mắt và miệng cho tính EAR và MAR ²².
- **Tính đặc trưng thời gian thực:** Dựa vào landmarks, tính toán các đặc trưng cốt lõi mỗi khung hình: EAR (giá trị tỉ lệ khía cạnh mắt), MAR (tỉ lệ hàm miệng), góc nghiêng/hạ cao đầu (qua landmarks mũi, mắt), trạng thái nhắm/mở của mắt (dựa trên EAR), trạng thái miệng (ngáp nếu MAR vượt ngưỡng). Có thể tính toán các giá trị trung bình trượt (ví dụ PERCLOS hoặc tần số chớp trung bình trong 1 phút).
- **Mô hình thời gian:** Vì buồn ngủ là hiện tượng chuỗi, thường cần gộp thông tin qua vài chục khung. Có thể đơn giản dùng smoothing (trung bình/Gaussian) và ngưỡng kết hợp (ví dụ báo động nếu trung bình EAR < ngưỡng trong N giây). Hoặc dùng mô hình CNN/LSTM nhẹ để học chuỗi. Ví dụ, một số công trình đề xuất dùng mạng 1D-CNN hoặc LSTM nhỏ nhận dãy EAR qua thời gian làm đầu vào. Cũng có thể dùng mô hình 3D-CNN (như [45#2]) trên chuỗi video, nhưng thường lớn.
- **Ngưỡng và cảnh báo:** Từ kết quả mô hình (xác suất buồn ngủ), sử dụng ngưỡng hoặc FSM (finite-state) để quyết định báo động. Cần hiệu chỉnh (calibration) cho từng cá nhân hoặc kịch bản để giảm cảnh báo sai. Ví dụ, nếu EAR trung bình trong 10s vượt ngưỡng ghi nhận buồn ngủ, thì phát âm thanh/cảnh báo.
- **Xử lý hậu kì:** Kết hợp các tín hiệu khác (ví dụ xe ngoạng lane, rung vô-lăng) nếu có, để giảm giả dương. Đảm bảo pipeline xử lý nhẹ, toàn xử lý trên thiết bị để bảo mật hình ảnh (privacy) ¹² ²³. Ví dụ, Bosch đã phát triển hệ thống nhúng trên xe phát hiện buồn ngủ bằng camera trên vô-lăng, xử lý tín hiệu mí mắt tại chỗ (không gửi dữ liệu ra xa) ²⁴ ²³.

Ví dụ: Ứng dụng thực tế như báo cáo [43] cho thấy dùng mô hình phát hiện chớp mắt trên OpenCV kết hợp điểm mốc Mediapipe để nhận dạng trạng thái ngáp đã đạt độ chính xác cao trong thử nghiệm. Họ nêu rõ “chúng tôi sử dụng phương pháp tỷ lệ EAR để phát hiện chớp mắt, kết hợp landmarks cho tình trạng ngáp” ⁹. Công trình [36] thì kết hợp đồng thời đầu cúi, hướng nhìn và nháy mắt, cho thấy một pipeline đa điểm tốt hơn chỉ dùng một loại đặc trưng.

4. Dữ liệu và huấn luyện

Dữ liệu công khai

- **UTA-RLDD** (Texas Arlington Real-Life Drowsiness Dataset): 180 video (~30 giờ) của 60 người ở ba trạng thái (tỉnh táo, cảnh giác thấp, buồn ngủ) ¹⁵ ²⁵. Đây là bộ dữ liệu lớn nhất, ghi lại các trường hợp buồn ngủ *vi tế* (mức 5 KSS) đến rõ (mức 10). Mỗi video ~10 phút, ghi bằng điện thoại ở nhiều góc, điều kiện và tự gán nhãn. Bộ này cho phép huấn luyện mô hình đa cấp cảnh báo dần.
- **NTHU-DDD** (National Tsing Hua University Drowsy Driving Dataset): Tập video hơn 9–10 giờ của ~36 người lái (độ tuổi và giới đa dạng), có cảnh ngày/đêm, có/nếu đeo kính, giả lập mệt (ngáp, nháy chậm, gật đầu) ²⁶ ¹. Có gắn nhãn từng khung về: mở/đóng mắt, nhịp gật đầu, hoạt động miệng (ngáp/đang nói) ¹. Bộ này chuẩn cho đánh giá trên các kịch bản khác nhau.
- **YawDD:** Dữ liệu ngáp lái xe do ĐH Ottawa phát triển (2014) ². Bao gồm hai bộ video, ghi yawn vs không yawn trong các điều kiện ánh sáng khác nhau, có nhiều người và góc camera. Tổng cộng ~952 video clip, cung cấp benchmark cho nhận dạng ngáp.
- **CEW (Closed Eyes in the Wild):** 2423 ảnh người thật với mắt mở/nhắm ²⁷. Phục vụ bài toán phát hiện mắt nhắm/ngắm; giúp huấn luyện sơ bộ cho tính EAR và nhận diện chớp.
- **Các dataset khác:**
 - *CEW & ZJU-eyeblick:* gồm ảnh mắt cho nhận dạng chớp/blink.
 - *TalkingFace, EmoReact* (tập biểu cảm mặt, có thể chuyển vào augmentation).

- *Self-collected (Field)*: ghi cảnh tài xế thực tế lái xe, có thể tự tạo tập số.

Phương pháp huấn luyện và tăng cường dữ liệu

- **Trước huấn luyện**: Trên bộ dữ liệu chung (ImageNet, MS-Celeb-1M) nếu dùng mạng chuẩn (MobileNet, SqueezeNet) để khởi tạo weights. Với mô hình tự thiết kế, khởi tạo Xavier/He.
- **Augmentation**: Rất quan trọng vì dữ liệu khó thu thập (trạng thái buồn ngủ hiếm). Các phép thường dùng: xoay nhỏ ($\pm 15^\circ$), dịch chuyển, thay đổi sáng/tối, nhiễu, làm mờ (blur), đeo kính/khăn (simulating occlusion), flip (nhưng xét không đổi tay lái). Sử dụng kỹ thuật tăng cường mạnh cho cảnh ban đêm hoặc điều kiện phức tạp (CLAHE, tăng tương phản) ²¹. Khuyến nghị mở rộng dữ liệu mặt người: ghép head pose từ library, mạng GAN (như StyleGAN thay đồ ngủ, hay Wav2Lip thêm chuyển động môi).
- **Gán nhãn và chuẩn hóa**: Nếu là bài học có giám sát, cần nhãn từng khung (tĩnh/ngáp), hoặc nhóm nhãn theo clip. Một số bộ ghi nhãn KSS (0-10), cần chuyển về 2 hoặc 3 lớp. Cân nhắc bình đẳng dữ liệu.
- **Huấn luyện mô hình**: Dùng cross-entropy (với lớp sigmoid/softmax), hoặc regression/năng suất trên thang ngưỡng. Có thể áp dụng học tăng cường (KB): dùng dropout, weight decay để tránh quá khớp (đặc biệt tập nhỏ). Nếu dùng kiến trúc CNN+RNN, huấn luyện từng phần (CNN rồi đến RNN).
- **Kiểm thử**: Chia tập train/test theo người (không có người trong train lặp lại ở test) để đảm bảo tính khái quát. Áp dụng *cross-validation* như RLDD khuyến nghị (5 folds) ²⁸. Kiểm thử trên kịch bản đa dạng (ánh sáng, ngáp thật vs giả).

Dữ liệu huấn luyện và tăng cường

Ví dụ cụ thể từ NTHU-DDD cho thấy pipeline thường là: phát hiện mặt (MTCNN/OpenCV), **landmarks**, crop vùng toàn mặt/đôi mắt/miệng, resize (thường 64×64 hoặc 128×128), chuẩn hóa (per-pixel hoặc CLAHE) ²¹. Nhiều công trình thiếu augmentation (họ chỉ trích là nhược điểm), nên nên thêm xoay nhỏ, jitter, lật, thay đổi độ sáng để nâng cao tính mạnh mẽ ²¹.

5. Kỹ thuật tối ưu hóa (<50k tham số)

Để đạt <50k tham số, cần kết hợp nhiều phương pháp:

- **Quantization (lượng tử hoá)**: Giảm độ chính xác trọng số xuống 8-bit (INT8) hoặc thấp hơn (4-bit, 1-bit). TensorFlow Lite QAT (quantization-aware training) hay post-training quantization đều phù hợp. Nghiên cứu *DDD TinyML* cho thấy sau lượng tử hóa động (DRQ), mạng CNN đơn giản còn ~50KB (khoảng 50k tham số) ³. Điều này giúp model cỡ vài chục KB có thể chạy trên MCU.
- **Pruning (cắt tỉa)**: Loại bỏ các trọng số nhỏ (unstructured) hoặc cả filter (structured). Ví dụ, prune 90% trọng số có thể giảm tham số mà ít giảm accuracy nếu kết hợp finetune sau. Có thể dùng công cụ TensorFlow Model Optimization Toolkit.
- **Weight sharing/clustering**: Chia trọng số thành cụm (clustering) và chia sẻ giá trị, giảm số lượng giá trị độc lập. TensorFlow Lite hỗ trợ weight clustering.
- **Kiến thức kết tinh (Distillation)**: Huấn luyện một mô hình *teacher* lớn, sau đó dùng đầu ra mềm (soft labels) để huấn luyện mô hình *student* nhỏ. Student (siêu nhỏ) có thể đạt được accuracy đáng kể nhờ học từ teacher.
- **Depthwise Separable Convolution**: Các mô hình như MobileNet tận dụng conv phân rã, giúp giảm tính toán/param. Mặc dù bản gốc vẫn >50k, nhưng ý tưởng dùng các convolution chỉ xử lý từng kênh riêng rẽ là bắt buộc để duy trì hiệu suất.
- **Binarization**: Các mạng nhị phân (như XNOR-Net) giảm trọng số xuống 1-bit; thực chất có thể xuống được ~60KB thậm chí với mạng nhỏ. Tuy nhiên, huấn luyện phức tạp, dễ mất accuracy.

- **NAS (Automated Search):** Có thể dùng công cụ NAS hoặc kiến trúc đề xuất (như MicroNet Challenge) để tự tìm kiến trúc tối ưu tham số/độ chính xác. Ví dụ các bài như OSPS-MicroNet (460k params) cho thấy hướng tới student CNN siêu nhỏ ²⁹.
- **TinyML Toolchains:** Sử dụng TensorFlow Lite Micro/Arm CMSIS-NN, ONNX Runtime Mobile, hoặc công cụ Edge Impulse để tối ưu mạng cho vi điều khiển. Các bộ tool này tự động lượng tử hóa/prune khi convert model.

Các kỹ thuật này có thể phối hợp: ví dụ, dùng NAS để chọn kiến trúc ước lượng 100k tham số, rồi prune còn 10–20k và quantize 8-bit. Trước khi deploy, luôn test lại độ chính xác sau tối ưu vì có thể giảm nhẹ.

6. Mục tiêu phần cứng và hiệu năng

Các thiết bị nhằm tới bao gồm:

- **ARM Cortex-A (điện thoại/board phát triển):** Ví dụ Raspberry Pi (Cortex-A72) có thể chạy model nhỏ (như MobileNet-V2 128px) ở ~30fps ³⁰. Với model ~50k param (64×64 input), dự kiến ~10–30fps (tùy ngôn ngữ); Bật NPU (như Google Coral) có thể lên 200+fps cho model 8-bit nhỏ.
- **Vi điều khiển (Cortex-M, ESP32):** STM32H7 (216MHz) hay ESP32 (240MHz, dual-core) chỉ có vài MB RAM. Trên STM32H7, với TensorFlow Lite Micro, một mạng ~20k tham số (tương đương vài 10KB) có thể cho ~20–50fps đối với ảnh đơn giản (xét tính toán đơn nhân). Ví dụ, model dạng MNIST (14k param) chạy ~10ms trên STM32H7. Với 50k param có thể lưu ý. Phải gom tensor và thâm hụt RAM (khoảng 5KB cho trọng số 8-bit). Một số MCU mới (Ambiq Apollo3, Synopsys ARC, NXP k64) cũng hỗ trợ DSP/NPU đơn giản.
- **NPU/TPU vi xử lý:** Thiết bị có NPU (Tensor processor) như Google Coral EdgeTPU, Nvidia Jetson Nano/TX2 NX hay smartphone (NN API) sẽ đẩy nhanh tốc độ đáng kể. Ví dụ, trên EdgeTPU, một mạng 8-bit ~20k param (<1ms) chỉ nhích một phần nhỏ do overhead truyền tải dữ liệu là chủ yếu. Trên Jetson Nano, các mạng CNN cỡ trung bình có thể ~50fps, nên microCNN <50k param chắc chắn xử lý trong vài ms.
- **FPGA/ASIC:** Nếu có thể, sử dụng mạng cứng (embedded CNN accelerator) sẽ đạt lợi thế. Nhưng phổ biến hơn là các lựa chọn trên.

Ví dụ tốc độ: Một báo cáo (Pytorch tutorial) cho thấy MobileNetV2 (320×320) chạy ~40fps trên Raspberry Pi 4 ³⁰. Mạng “TinyCNN” (50k param, 64×64) ước tính nhẹ hơn >>60fps. Trên ESP32, mạng chữ ký âm thanh (15k param) đạt ~20ms, nên kỳ vọng mạng 40k–50k param tương tự chục ms.

7. Ghi chú triển khai và kiến trúc mẫu

- **Triển khai:** Dùng thư viện TensorFlow Lite (tập lệnh C++ cho MCU: TFLite Micro), hoặc ONNX Runtime (có bản cross-compile cho ARM). Môi trường phát triển như Edge Impulse hỗ trợ nhanh từ dữ liệu ra firmware. Arduino cũng có addon TensorFlow Lite. Cần chú ý sinh code inference tối ưu (chọn quantized kernel, hạn chế dynamic allocation).
- **Mẫu kiến trúc:** Ví dụ đơn giản: một mạng CNN 3 lớp (Conv(8)->ReLU->Pooling-> Conv(16)->ReLU->Pooling-> FC(2) hoặc FC(16)->softmax) có thể vào tầm 30–40K tham số với input ~64×64. Hoặc Conv(16×3×3)->ReLU->DWConv(32×3×3)->ReLU->FC(2)... kết hợp Dropout nếu cần. Ví dụ minh họa:

```
Input 64x64x1
├ Conv(8 filters, 3x3) + ReLU
├ DepthwiseConv(8 filters, 3x3) + ReLU
└ MaxPool(2x2)
```

```

└ Conv(16 filters, 3x3) + ReLU
└ DepthwiseConv(16 filters, 3x3) + ReLU
└ MaxPool(2x2)
└ FC(16) + ReLU
└ FC(2) + Softmax (drowsy/alert)

```

Cấu trúc này khoảng 30–40k tham số khi gộp FC, phù hợp MCU. Cũng có thể thay Conv + DWConv bằng 2 Conv nhỏ liên tiếp.

- **Code/Tools:** Google TFLite Micro, Arm CMSIS-NN (cung cấp bản optimized cho ARM), ONNXRuntime, Edge Impulse (tích hợp sẵn nhiều mô hình mẫu). Thư viện MediaPipe FaceMesh để landmark nhẹ (có bản C++). Chọn framework có hỗ trợ lượng tử hóa (Int8) để tích hợp vào MCU.
- **Bảng so sánh:** Nêu ở phần đầu đã tổng hợp một số kiến trúc gợi ý. Có thể cập nhật thêm nếu tìm thấy các mô hình siêu nhỏ khác (ví dụ “MicroNet: Improving Image Recognition with extremely low FLOPs”³¹) nhưng hiện chưa có phiên bản <50k cụ thể công bố.

8. Rủi ro, lỗi và đạo đức

- **Sai sót/giới hạn:**
 - *Sai cảnh báo giả:* Ví dụ người tài xế nhắm mắt vì dừng đèn đỏ, hoặc khuôn mặt che chắn (tay bịt mặt). Hệ thống cần tránh báo động sai thường xuyên.
 - *Không nhận ra:* Cảnh buồn ngủ tinh vi (vd. *microsleep* vài giây) rất khó phát hiện chỉ qua hình ảnh; dấu hiệu có thể không rõ ràng đến khi quá muộn.
 - *Phụ thuộc môi trường:* Ánh sáng yếu, camera gắn lệch, tài xế nhìn đi chỗ khác không phải dấu hiệu buồn ngủ nhưng có thể làm model nhầm.
 - *Thiết bị biên hạn chế:* Model dưới 50k tham số dù được tối ưu rất kỹ vẫn có độ chính xác giới hạn. Cần kết hợp cảm biến khác (âm thanh tí tí từ xe, sinh trắc học) để tăng độ tin cậy.
 - *Giới hạn training:* Đa số bộ dữ liệu là giả lập hoặc thu trong phòng thí nghiệm; hiệu quả thực tế trên tài xế thật (nhiều chủng tộc, ngoại hình, tư thế lái) có thể kém hơn.
- **Vấn đề đạo đức/riêng tư:**
 - **Riêng tư:** Xử lý hoàn toàn *on-device* (đặc biệt đối với camera nhận diện khuôn mặt) để bảo vệ quyền riêng tư. Không lưu ảnh hoặc gửi lên máy chủ. Ví dụ hệ thống Bosch ghi rõ dữ liệu chỉ dùng local, không chia sẻ²⁴.
 - **Sự chấp nhận:** Tài xế cần được thông báo rõ khi camera giám sát. Cần tránh cảm giác bị giám sát quá mức.
 - **Phán đoán cá nhân:** Mức độ buồn ngủ phụ thuộc cá nhân (tuổi, sức khỏe, cường độ làm việc). Cần chính sách huấn luyện công bằng (gồm đa dạng độ tuổi, giới, điều kiện sức khỏe).
 - **Giới hạn về công bằng:** Model ảnh hưởng độ sáng, màu da có thể hoạt động kém trên một số người (VD: da tối, thiếu sáng); cần đánh giá fairness.
 - **Trách nhiệm:** Cảnh báo giả có thể khiến người dùng bỏ qua (desensitization). Cũng có thể dẫn tới trách nhiệm pháp lý nếu tài xế cho là cảnh báo sai.

Tóm lại, phải triển khai cẩn thận, kiểm thử kỹ trên dữ liệu thực tế và cân nhắc kỹ ở giai đoạn thử nghiệm. Hệ thống chỉ nên đóng vai trò “hỗ trợ” cảnh báo; tài xế vẫn phải giữ trách nhiệm lái xe.

Tài liệu tham khảo chính

- VnExpress/Báo Bắc Ninh – Tin tức về công nghệ buồn ngủ (Bosch) ²⁴ ²³ .
- Zhang et al. 2022 (MDPI): *Comparison of Eye and Face Features on Drowsiness Analysis* ¹⁹ ¹⁶ .
- F. Kouveli et al. 2024 (MDPI): *Real-Time Fatigue Detection Algorithms* (CNN, yawning) ¹⁹ ⁵ .
- Abdelaziz et al. 2023 (*DDD TinyML* – Sensors): mô hình TinyML cho buồn ngủ ³ .
- Abe 2023 (*Sleep Advances*): nghiên cứu PERCLOS và drowsiness ⁶ .
- Tashakori et al. 2022: Thermal imaging cho buồn ngủ ¹⁷ ¹⁸ .
- Weng et al. 2017 (poster): Drowsiness detection bằng Facial Action Units ¹³ ¹⁴ .
- Choi & Kim 2015: Phối hợp head pose, gaze, blink ¹² ¹¹ .
- Dattani et al. 2022: *Real-Time Drowsiness Detection (Sensors)* – pipeline chớp mắt và ngáp ⁹ .
- UTA-RLDD Dataset (Reza Ghoddoosian) ¹⁵ ²⁵ .
- NTHU-DDD Dataset (EmergentMind) ³² ¹ .
- YawDD Dataset (Ottawa) ² .
- CEW Dataset (Nanjing UA) ²⁷ .

¹ ²¹ ²⁶ ³² NTHU-DDD: Driver Drowsiness Dataset

<https://www.emergentmind.com/topics/nthu-ddd-dataset>

² YawDD: A Yawning Detection Dataset - QUALINET Databases

https://qualinet.github.io/databases/video/yawdd_a_yawning_detection_dataset/

³ ⁴ DDD TinyML: A TinyML-Based Driver Drowsiness Detection Model Using Deep Learning - PMC

<https://pmc.ncbi.nlm.nih.gov/articles/PMC10305041/>

⁵ Eye Aspect Ratio for Real-Time Drowsiness Detection to Improve Driver Safety | MDPI

<https://www.mdpi.com/2079-9292/11/19/3183>

⁶ PERCLOS-based technologies for detecting drowsiness: current evidence and future directions - PMC

<https://pmc.ncbi.nlm.nih.gov/articles/PMC10108649/>

⁷ ⁸ ⁹ ¹⁰ ²² Real-Time Deep Learning-Based Drowsiness Detection: Leveraging Computer-Vision and Eye-Blink Analyses for Enhanced Road Safety - PMC

<https://pmc.ncbi.nlm.nih.gov/articles/PMC10384496/>

¹¹ ¹² naturalspublishing.com

<https://www.naturalspublishing.com/download.asp?ArtcID=8300>

¹³ ¹⁴ ailb-web.ing.unimore.it

<https://ailb-web.ing.unimore.it/icpr/media/posters/12108.pdf>

¹⁵ ²⁵ ²⁸ UTA-RLDD

<https://sites.google.com/view/utarlidd/home>

¹⁶ ¹⁹ ²⁰ Comparison of Eye and Face Features on Drowsiness Analysis - PMC

<https://pmc.ncbi.nlm.nih.gov/articles/PMC9460799/>

¹⁷ ¹⁸ Driver drowsiness detection using facial thermal imaging in a driving simulator - PubMed

<https://pubmed.ncbi.nlm.nih.gov/34477030/>

²³ Công nghệ AI giúp cảnh báo buồn ngủ khi lái xe - Báo VnExpress

<https://vnexpress.net/cong-nghe-ai-giup-canh-bao-buon-ngu-khi-lai-xe-4029264.html>

²⁴ Công nghệ AI giúp cảnh báo buồn ngủ khi lái xe

<https://baobacninhvt.vn/cong-nghe-ai-giup-canh-bao-buon-ngu-khi-lai-xe.bbg>

27 CEW closed-eye Face Dataset | Datasets | HyperAI

<https://hyper.ai/en/datasets/9161>

29 OSPS-MicroNet: a distilled knowledge micro-CNN network for ...

<https://www.frontiersin.org/journals/computer-science/articles/10.3389/fcomp.2024.1279810/full>

30 Real Time Inference on Raspberry Pi 4 and 5 (40 fps!)

https://docs.pytorch.org/tutorials/intermediate/realtime_rpi.html

31 [PDF] MicroNet: Improving Image Recognition with Extremely Low FLOPs

http://www.svcl.ucsd.edu/projects/micronet/assets/MicroNet_ICCV2021.pdf